

Speeding Up the Web with Rust and WebAssembly: Evaluating Single-Threaded and Rayon-Based Multithreaded Performance

Parth Bathe
Dept. Of Computer Applications
Dr. Vishwanath Karad
MIT World Peace University.
Parth.bathe0142@gmail.com

Siddhant Wadhai
Dept. Of Computer Applications
Dr. Vishwanath Karad
MIT World Peace University.
Siddhantwadhai20@gmail.com

ABSTRACT

WebAssembly (Wasm) is a low-level, portable compilation target designed to deliver near-native execution performance inside modern web browsers. For a long time, JavaScript's strictly single-threaded model was a fundamental limitation, one that made it difficult to make compute-heavy applications. The introduction of threading support via SharedArrayBuffer changed that, making in-browser parallelism not just theoretically possible, but practically worth exploring. This paper evaluates the performance of Rayon-based multithreaded WebAssembly compiled from Rust against single-threaded Wasm and native JavaScript across six benchmark tasks: summation, matrix multiplication, image blurring, substring search (grep), sorting, and array correlation. Benchmarks were executed across three major browsers (Firefox, Chrome, and Microsoft Edge) with 8 spawned threads. Results show that multithreaded Wasm achieves substantial speedups (up to 8) for compute-intensive, embarrassingly parallel workloads such as matrix multiplication and image processing, while exhibiting no benefit or negative returns for memory-bandwidth-bound and string-heavy tasks. The sum benchmark revealed LLVM's compile-time constant folding, reducing median execution to zero. High coefficients of variation were observed uniformly, consistent with the non-determinism of browser JIT environments and the precision limitations of performance.now(). Taken together, these results show that Rayon-based multithreading in Wasm is a legitimate option for arithmetic-heavy computation on the client side, though it is not a blanket solution, and workload characteristics matter more than one might initially assume.

Keywords: *WebAssembly, Rust, Rayon, multithreading, browser performance, benchmarking, wasm-bindgen*

I. INTRODUCTION

For most of the internet's history JavaScript has been the language for doing work on the client side. Its single threaded event loop model designed to make programming easy and avoid problems with multiple tasks happening at the same time as worked well for tasks that wait for input or output. As web apps have gotten more demanding, tasks like image processing, physics simulations and data analysis have started to slow down the main thread. This makes the user experience worse and the app less responsive.

Wasm [1] was created to solve this problem. It is a bytecode format that lets code written in low-level languages like Rust, C and C++ run in the browser almost as fast as native code. Wasms execution model is sandboxed, which means it is expected to provide the safety guarantees and predictable performance that JavaScript cannot match.

Among languages that can be compiled to Wasm Rust stands out because of its tools. The wasm-bindgen crate [2] helps exchange data between Rust and JavaScript. The Rayon crate[3] provides a high-level API for data processing. And finally, the wasm-bindgen-rayon crate [4] helps tie them together and spawn worker threads for use. When combined these tools let Rust code use threads in the browser. Despite the potential of Wasm, its performance is not well understood. This study evaluates the performance of native JavaScript, single threaded Wasm and multithreaded Wasm. We test them on six tasks with varying levels of complexity. Our goal is to determine when multithreaded Wasm provides a speedup and when its overhead outweighs the benefits. This will help developers decide whether to use this approach for developing performant web apps.

II. BACKGROUND AND RELATED WORK

A. Evolution of WebAssembly

Before WebAssembly, high-performance code in the browser mainly relied on asm.js [5], which is a strict and statically typed subset of JavaScript. It was typically generated from languages like C and C++ using Emscripten [6]. Although asm.js made ahead-of-time (AOT) optimizations possible, the code still had to run through the JavaScript engine. This introduces extra overhead and often leads to larger code sizes.

Wasm was introduced in 2015 as a more efficient and language-independent alternative to asm.js, supported by all major browsers. It uses a compact binary format, which helps in faster loading and execution. Wasm is based on a stack machine, designed with performance, safety, and portability in mind. Its first stable version (MVP) was released in 2017.

Since then, WebAssembly has continued to evolve. It now includes support for features [7] such as floating-point operations, multithreading, reference types, SIMD instructions, and exception handling. Wasm is also being used in areas like serverless computing, edge computing, and embedded systems through different runtimes such as Wasmtime [8] and Wasmer [9][10].

B. Comparison with Java Bytecode

WebAssembly and Java bytecode are both designed as portable virtual instruction formats, but they follow different design approaches. Java bytecode is meant to run in a managed environment that includes features like garbage collection, class loading, and a large standard library. It typically relies on long-running processes that use Just-In-Time (JIT) compilation for optimization to improve code as it runs.

In contrast, WebAssembly focuses on fast startup, predictable performance, and strong sandboxing, especially for running code in untrusted environments like browsers[11]. It avoids builtin high-level features such as objects or dynamic dispatch, leaving these responsibilities to the source language and its compiler.

This design allows compilers like LLVM to generate smaller, highly optimized code tailored to specific tasks, instead of relying on a heavy runtime system.

C. Multithreading on the Web

Traditionally, web applications can use **Web Workers** [12] (introduced in 2009) for parallel execution. These workers run separately and communicate through message passing. While this works well for independent tasks, it becomes inefficient for fine-grained parallelism because data needs to be copied between workers, which can be slow for large quantities of data[3].

WebAssembly improves this through its Threads proposal, which introduces shared memory. Using **SharedArrayBuffer** [13], multiple Web Workers can access the same memory space within a single Wasm module. This allows more efficient parallel processing without constant data copying.

Synchronization between threads is handled using atomic operations defined in the Wasm specification. Libraries like **wasm-bindgen-rayon** make this easier by connecting Rust's Rayon thread pool to WebAssembly's threading model.

However, enabling multithreading in WebAssembly requires cross-origin isolation for security reasons. Servers must include specific HTTP headers:

Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp

to prevent side-channel attacks such as Spectre [14]. While this adds some constraints, it can be mitigated by isolating the use of multithreaded wasm to only a few pages in an application where it is needed.

III. METHODOLOGY

A. Research Objective

The primary objective of this study is to characterize the performance of three execution models native JavaScript, single-threaded Rust-compiled WebAssembly, and 8-thread Rayon-based multithreaded WebAssembly across a representative selection of computational workloads. Performance is measured as wall-clock execution time using the browser's Performance API. The thread count is fixed at 8 for all multithreaded experiments in order to isolate the cost-benefit trade-off of introducing multithreading in WebAssembly. While performance is expected to vary with different thread counts, analyzing scalability across varying levels of parallelism is beyond the scope of this study.

Secondary objectives include characterizing execution time variability across browser environments, identifying the mechanisms responsible for observed performance differences, and producing practical guidance for developers selecting execution strategies for client-side compute workloads.

B. System Architecture

The benchmarking framework is a minimal client-server application designed to ensure high measurement accuracy while reducing unnecessary overhead and external interference.

Frontend:

The frontend is a single-page application implemented in plain HTML5, CSS3, and vanilla JavaScript, avoiding any frontend framework overhead. Benchmark runs are triggered via button controls or through an automation script. Timing is measured using `performance.now()`. All Wasm module initialization (including thread pool setup) occurs before benchmark timing begins, ensuring that measured times reflect only the execution of the benchmark function itself.

Backend:

A lightweight Node.js server serves the static frontend assets and Wasm modules. Its primary role beyond file serving is to attach the HTTP response headers required for cross-origin, which are mandatory for *SharedArrayBuffer* access and hence for the multithreaded Wasm benchmarks. The backend also stores benchmark results to an sqlite database for analysis and generates reports.

C. Benchmark Implementation

All benchmarks are implemented in three variants, ensuring strict algorithmic consistency across different execution models:

The *singlethread* Rust crate contains conventional, sequential Rust implementations compiled to single-threaded Wasm using *wasm-pack*. No parallel primitives are used; these implementations are structurally identical to the native Rust code from which they are compiled.

The multithreaded crate implements the same algorithms using Rayon's parallel iterators and parallel sorting, and is compiled with *wasm-bindgen-rayon* to enable threading via Web Workers. Enabling the Rayon feature flag activates parallel execution, although a sequential fallback exists and is not used in these benchmarks.

Each benchmark is additionally implemented in native JavaScript, ensuring that the JavaScript baseline uses idiomatic, performant code (typed arrays, native string methods, standard library sort) rather than a naive transcription of the Rust algorithm.

Each variant is implemented without any specific performance tuning to match code that would generally be found in any web frontend, ensuring that the findings are applicable to an average app that isn't already highly optimised.

D. Benchmark Descriptions

1) Sum (Reduction):

The sum benchmark computes the total of a fixed, compile-time constant array. It is designed to evaluate compiler optimization behavior rather than runtime performance. JavaScript implements this using a simple for-loop, while Rust uses an iterator-based sum.

2) Matrix Multiplication:

This benchmark performs dense matrix multiplication on two $n \times n$ matrices with f64 elements. Each output element $c[i][j]$ requires n multiply-accumulate operations, resulting in a total computational complexity of $O(n^3)$. The Rust multithreaded implementation uses Rayon's `par_iter()` to parallelize computation across output rows, while the JavaScript and singlethreaded wasm uses a triply nested loop. Input ranges from 20 to 400 height and width.

3) Image Blur:

This benchmark applies a Gaussian blur to an RGBA image, where each output pixel is computed as the weighted average of its 3×3 neighborhood. The total computational complexity is $O(n^2)$ for an $n \times n$ image. The multithreaded Rust implementation distributes row bands across Rayon threads, while the JavaScript version operates on a `Uint8ClampedArray`. Input ranges from 50px to 1000px height and width.

4) Grep (Substring Search):

Grep performs substring search for a fixed pattern within paragraph-like strings of varying length. JavaScript uses the native `String.includes()` method, which operates on V8's internal UTF-16 string representation. In contrast, Rust uses `str::contains()`, operating on a UTF-8 byte slice within Wasm linear memory. The difference in string format is important here, and adds significant overhead

that reverses the expected results. Input strings contain 100 to 1000 lines each of length around 100 characters.

5) *Sort:*

Evaluates the parallel sorting of a floating-point array. All implementations use the built in stable sort algorithms that would normally be used in any application. Unlike other tests that can take a reference, the array generated in js has to be copied over to wasm linear memory for processing and this adds overhead that increases with size, even more for MT where chunks of the array need to be copied to multiple web workers. Input ranges from 100 to 3000 elements.

6) *Correlation:*

Computation of the Pearson correlation coefficient between two f64 arrays. The algorithm requires two linear passes (for means) and one pass for the covariance and standard deviations, giving $O(n)$ time complexity. Rayon's parallel iterator partitions the summation across threads. Input ranges from 1000 to 30000 pairs of f32.

E. Execution Environment

All experiments were conducted on a single machine running Fedora Linux 43 (Workstation Edition) with an 13th Gen Intel® Core™ i5-13420H $\times$ 12 processor (8 performance cores, and 16 GB DDR4 RAM. Browsers tested: Firefox 140, Chrome 146, and Microsoft Edge 146. All multithreaded tests were fixed at 8 threads, matching the available core count. Background applications were minimized to reduce scheduling noise, though the machine was not fully isolated and some noise is still present in the results, though they do not change the general conclusions drawn.

F. Measurement Procedure

Each benchmark task browser and input combination was run a 100 times as warmup, before measuring 50 runs with the same input parameters for each technology

Wasm modules are loaded and thread pools initialized once before all timing runs begin; module initialization time is excluded from measurements. For each run, `performance.now()` is called immediately before and after the benchmark function invocation, and the difference is recorded. Warmup runs of each implementation are

executed before the measured 50 runs to ensure JIT activities do not affect the results.

Median execution time across the 50 runs after IQR filtering is reported due to its robustness to the skewed distributions that are common in browser execution environments (outlier runs caused by JIT deoptimization, GC pauses, or OS interrupts). The coefficient of variation is reported alongside median times to show predictability.

It is important to note that although `performance.now()` provides high-resolution timestamps, its effective precision is intentionally reduced in modern browsers for security and privacy reasons (such as to mitigate timing attacks like Spectre). The actual resolution varies across browsers, typically on the order of $\sim 5 \mu s$ in Chromium-based browsers (e.g., Chrome, Edge) under ideal conditions, and coarser (often $\sim 0.1 ms$ or more) in Firefox depending on privacy settings. Additionally, timer jitter and clamping may be applied by the browser. As a result, very short-running benchmarks may be subject to quantization errors, so small differences between implementations should be interpreted with caution. This limitation is partially mitigated in this study by aggregating results over multiple runs.

IV. RESULTS

A. Summary Table

Table I summarizes the median execution times and thread speedup ratios across all evaluated browsers and benchmarks. The reported medians are computed over multiple runs to reduce the impact of outliers and presents stable performance characteristics.

Thread speedup is defined as the ratio of the single-threaded Wasm median execution time to the multithreaded Wasm median execution time, as in a measure of the performance gain achieved through parallelism. A higher value indicates more effective utilization of multiple threads and better performance improvements.

The benchmark 'sum' is not reported in this table as it is an exceptional test that has no input parameter and has special behaviour in wasm.

TABLE I. BENCHMARK RESULTS: MEDIAN EXECUTION TIME (ms), CV(%), WASM OVERHEAD THREAD SPEEDUP AND PARAMETER RANGE

browser	test	native md	single md	multi md	native cv	single cv	multi cv	single oh	multi oh	thread speedup	param min	param max
Firefox	matrix	20.98	18.97	2.31	109.8196	113.5828	109.4624	0.9042	0.1101	8.2121	20	400
Firefox	image	13.205	6.41	1.755	77.8393	84.3191	74.4212	0.4854	0.1329	3.6524	50	1000
Firefox	grep	0.28	0.74	1.1	39.4655	35.6887	26.7782	2.6429	3.9286	0.6727	100	1000
Firefox	sort	0.5	0.25	0.37	58.489	44.1153	58.2392	0.5	0.74	0.6757	100	3000
Firefox	correlation	1.73	0.19	0.46	47.9414	44.4444	12.0494	0.1098	0.2659	0.413	1000	30000
Chrome	matrix	23.8113	18.0825	2.1475	112.5944	114.5707	105.6969	0.7594	0.0902	8.4203	20	400
Chrome	image	8.07	5.84	1.85	82.8846	84.8375	69.8477	0.7237	0.2292	3.1568	50	1000
Chrome	grep	0.14	0.595	0.86	23.3898	21.4981	9.2415	4.25	6.1429	0.6919	100	1000
Chrome	sort	1.1325	0.2275	0.3425	57.0193	40.5916	58.4067	0.2009	0.3024	0.6642	100	3000
Chrome	correlation	1.7325	0.1425	0.4775	45.939	38.2787	12.8802	0.0823	0.2756	0.2984	1000	30000
Edge	matrix	23.845	18.6475	2.555	112.8973	113.9336	102.5203	0.782	0.1072	7.2984	20	400
Edge	image	8.0975	6.165	2.0925	83.6613	85.0976	69.8117	0.7613	0.2584	2.9462	50	1000
Edge	grep	0.11	0.47	0.795	20.1366	19.3316	9.6363	4.2727	7.2273	0.5912	100	1000
Edge	sort	1.1025	0.265	0.3587	56.9921	41.8402	58.5032	0.2404	0.3254	0.7387	100	3000
Edge	correlation	1.7125	0.15	0.4675	45.3608	37.861	11.0057	0.0876	0.273	0.3209	1000	30000

B. Sum Benchmark: LLVM Constant Folding

The sum benchmark produced a median execution time of exactly 0 ms (+ function call overhead and measurement jitter) for single-threaded Wasm across all three browsers, while native JavaScript executed in 445–528 ms. Upon inspection of the WAT file it was found that the function body was replaced with:

```
(func (;89;) (type 25) (result i64)
  i64.const 499999999500000000
)
```

This reflects LLVM's compile-time constant folding: because the Rust sum function operates on a compile-time-known array with no runtime parameters, LLVM evaluates the result at compile time and emits a Wasm module that simply returns the precomputed value. No summation loop executes at runtime.

This demonstrates a structural asymmetry between AOT and JIT compilation. LLVM can perform whole-function static analysis at compile time, including constant propagation across the entire function body. V8's JIT operates on runtime values only and cannot see the compile-time constants. Multithreaded Wasm shows non-zero times (~66–80 ms) because spawning the Rayon thread pool via Web Workers incurs real initialization overhead regardless of computation triviality. Figures 1–3 illustrate this three-point scatter for all browsers.

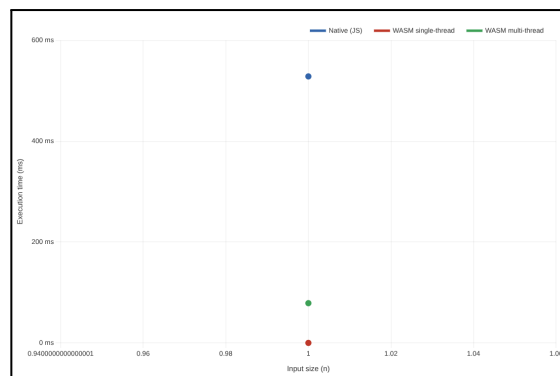


Fig. 1. Sum Chrome. Single-thread Wasm reaches 0 ms via LLVM constant folding; native JS executes the full 528 ms summation loop.

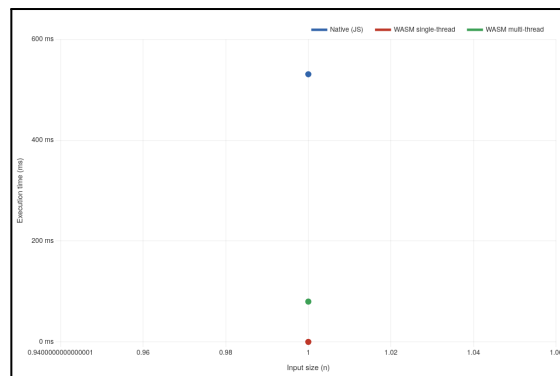


Fig. 2. Sum Firefox. Same pattern as Chrome: single-thread Wasm at 0 ms, native JS at ~445ms, multi-thread at ~66ms.

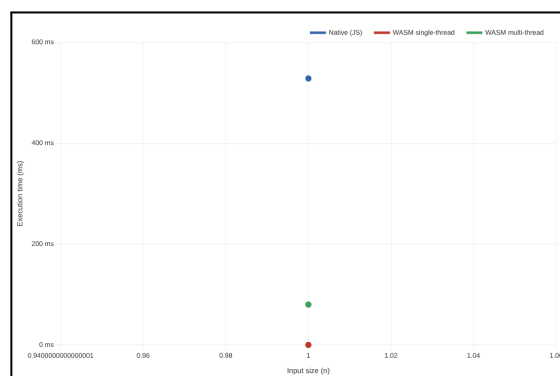


Fig. 3. Sum Microsoft Edge. Consistent with other browsers; single-thread Wasm at 0 ms, multi-thread at ~80 ms.

C. Matrix Multiplication: Near-Linear Parallel Scaling

Matrix multiplication is the benchmark most favorable to multithreaded Wasm. Chrome achieves 7x-8x speedup for all browsers, all close to $8\times$ speedup from 8 threads. These strong results are expected: each output row is entirely independent (no coordination needed), arithmetic intensity is high (n multiplications per element), the data layout is cache-friendly for row-major partitioning, and the f64 array can be passed as a zero-copy pointer into Wasm linear memory.

The growth curves in Figures 4-6 clearly show multithreaded Wasm (green) diverging sharply from native JS (blue) and single-thread Wasm (red) as input size n grows, with the gap widening as $O(n^3)$ computation dominates.

Interestingly, native JS performance differs between engines despite identical conditions. Firefox (with SpiderMonkey) reaches approximately 145ms at $n=400$, while Chrome (with V8) reaches approximately 168ms, a gap of around 15%. The consistency of this difference across input sizes, combined with near-identical single-threaded Wasm times between the two browsers, indicates the gap originates in JIT code quality in js rather than environmental noise. Matrix multiplication's triply-nested loop over typed numeric arrays is precisely the pattern that SpiderMonkey's WarpMonkey optimising tier handles aggressively, benefiting from Mozilla's long investment in numeric loop optimisation that traces back to the asm.js era. V8's TurboFan, while highly optimised for general JavaScript workloads, leaves a comparatively larger gap between its native JS and single-threaded Wasm performance roughly 38 ms at $n=400$ versus only 13 ms for SpiderMonkey, suggesting that SpiderMonkey's JIT is emitting tighter numeric code for this access pattern and getting meaningfully closer to bare Wasm throughput leaving lesser room for improvement.

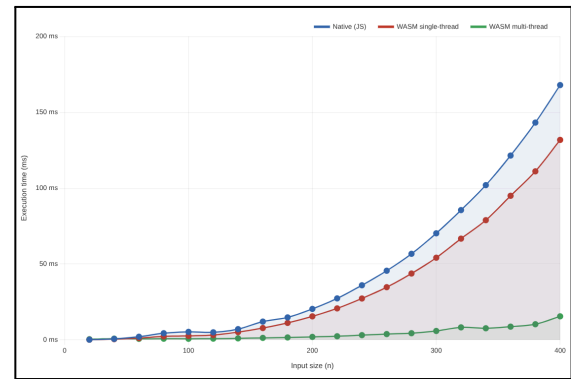


Fig. 4. Matrix multiplication Chrome. Multi-thread Wasm (green) diverges sharply from native JS and single-thread Wasm as n grows.

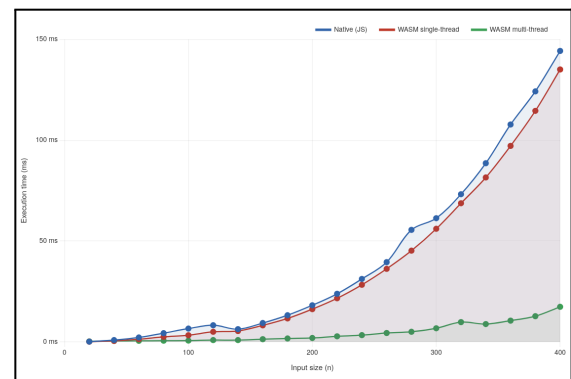


Fig. 5. Matrix multiplication — Firefox. Note near-identical curvature of native JS and single-thread Wasm lines, with multi-thread well below both throughout the input range.

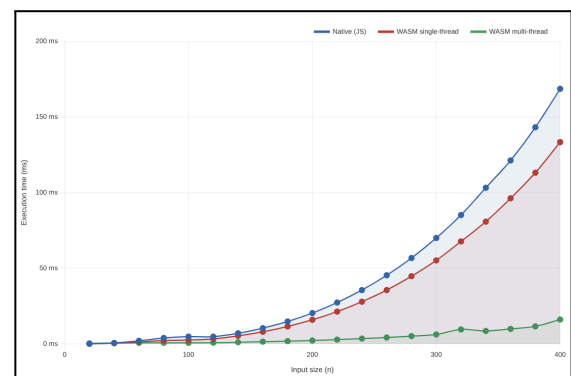


Fig. 6. Matrix multiplication — Microsoft Edge. Matches Chrome.

D. Image Blur: Good Parallel Efficiency with Memory Pressure

Image blurring shows consistent speedups of around 3x for multithreaded Wasm. The lower speedup compared to matrix multiply reflects greater memory traffic: each output pixel reads a 3×3 neighborhood (36 bytes) and writes 4 bytes, increasing cache pressure at larger input sizes. Boundary pixels at thread band edges require access to adjacent threads' rows, introducing minor cache-line sharing.

Figures 7–9 show the three curves growing quadratically as expected for $O(n^2)$ blur, with multi-thread (green) consistently below both other lines. The gap between native JS and single-thread Wasm is most pronounced in Firefox, not because Wasm is faster there, but because JavaScript performance is slower compared to Chrome and Edge. In contrast, both single-threaded and multithreaded Wasm exhibit nearly identical execution times across all browsers. This indicates that the larger observed gap in Firefox is caused by differences in JavaScript engine performance rather than improvements in Wasm execution. Notably, this contrasts with the earlier matrix multiplication benchmark, where Firefox's JavaScript outperformed Chrome. The discrepancy can be explained by workload characteristics: matrix multiplication is compute-bound and benefits from strong JIT optimization of arithmetic-heavy loops, whereas image blurring is memory-bound, dominated by frequent typed array accesses and cache behavior. As a result, Firefox's strength in compute-heavy scenarios does not translate to this memory-intensive workload, where Chrome and Edge handle JavaScript more efficiently.

Although medians over 50 runs were used to reduce noise, 2-3 visible bumps appear in the blur curves that are absent in the matrix multiplication results. This is a measurement artefact of scale: blur execution times are substantially lower in absolute terms, meaning that any noise which would be absorbed as rounding noise in a 100 ms measurement constitutes a noticeable percentage deviation in a 5-10 ms one. These bumps should not be interpreted as algorithmic irregularities. Similar bumps are present in all subsequent tests, which have even shorter execution times making them more visible.

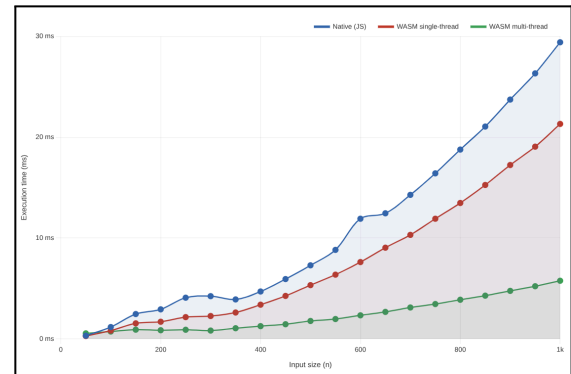


Fig. 7. Image blur Chrome.

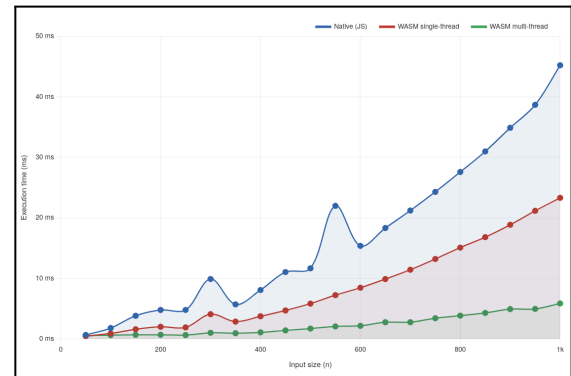


Fig. 8. Image blur Firefox. Single-thread Wasm (red) is faster here, indicating SpiderMonkey's Wasm image loop code generation outperforms V8 for this workload.

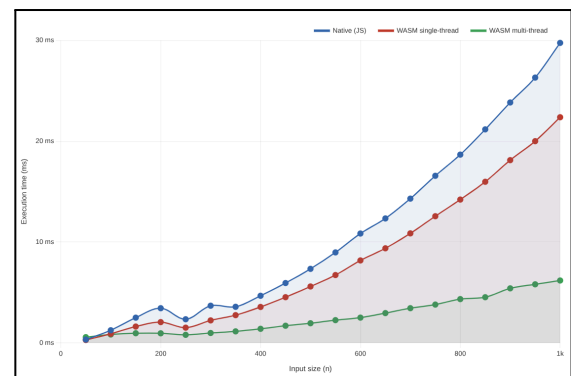


Fig. 9. Image blur Microsoft Edge. Pattern mirrors Chrome.

E. Grep: String Boundary Cost Inverts Expected Performance

The grep benchmark produces interesting results: native JavaScript (blue) is the fastest, followed by single-thread Wasm, with multi-thread Wasm being the slowest, the opposite of what one might expect from compiled code. The problem is the JS to Wasm string transcoding boundary. JavaScript engines store strings in UTF-16 while Rust's `&str` requires UTF-8 in Wasm linear memory. On every call, wasm-bindgen must transcode and copy the entire string into Wasm memory, then free it on return. V8's `String.includes()` operates directly on its internal heap representation with no copies.

Figures 10-12 clearly visualize this inversion: the blue native JS line sits consistently at the bottom, the red single-thread Wasm line in the middle, and the green multi-thread line at the top. All three lines grow roughly linearly with string length ($O(n)$), but the absolute overhead offsets differ by the boundary cost. Multi-thread adds thread coordination overhead on top of the already-expensive transcoding. The inversion is consistent and reproducible across all three browsers, confirming this is architectural rather than engine-specific.

One possible mitigation is to replace string-typed boundaries with Rust-exported enums wherever strings are used as a discriminant or pattern identifier rather than arbitrary text. Wasm-bindgen serialises `#[wasm_bindgen]` enums as plain integers across the boundary, eliminating transcoding entirely. This is effective when the set of patterns is finite and known at compile time, but cannot help when the data is genuinely arbitrary input.

In workloads where the same string is operated upon multiple times, it can be transferred once into wasm memory and then accessed without repeatedly copying it across the boundary, reducing the effect of the overhead and possibly being better in a very long run.

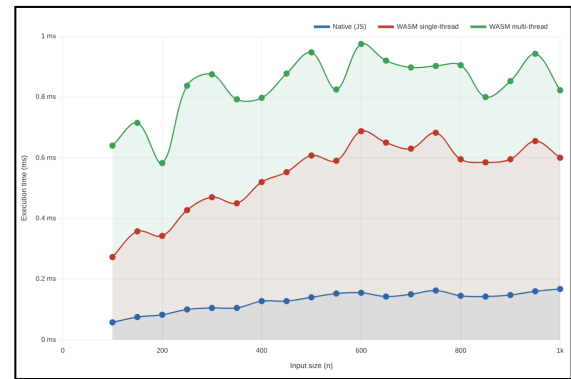


Fig. 10. Grep (substring search) Chrome. Line order inverted from expected: native JS (blue) fastest, then single-thread Wasm (red), then multi-thread Wasm (green) slowest throughout the input range due to UTF-16→UTF-8 transcoding per call.

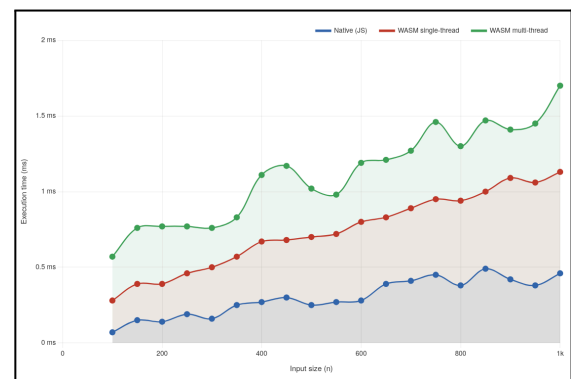


Fig. 11. Grep Firefox. Same inversion as Chrome.

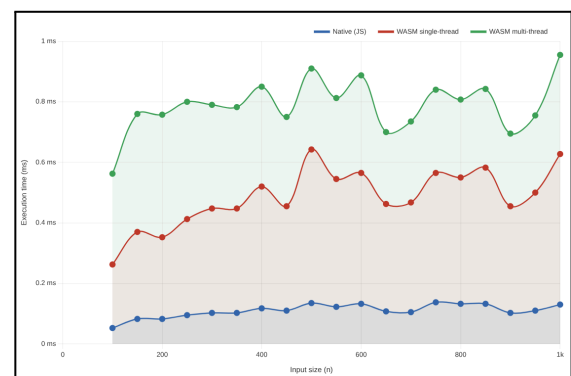


Fig. 12. Grep Microsoft Edge. Inversion consistent with Chrome and Firefox.

F. Sort: Coordination Overhead Dominates at Small Input Sizes

The sort benchmark shows multithreaded Wasm performing slightly worse than single-threaded Wasm across all browsers. This is explained by the relationship between input size and Rayon's parallel sort break-even threshold. Rayon's parallel sort is a parallel merge sort variant: it recursively partitions the input, sorts sub-slices in parallel, and merges. The merge phases require sequential coordination. For small inputs, the sequential merge dominates, and Rayon typically breaks even with sequential sort only around 100,000–500,000 elements far above the tested range of 50–3,000.

All three implementations use TimSort as their underlying algorithm: V8, SpiderMonkey, and Rust's slice::sort all default to a stable TimSort with insertion sort for small runs. This means performance differences between engines are purely a quality-of-implementation story rather than an algorithmic one, with SpiderMonkey's TimSort running measurably faster, consistent with SpiderMonkey's stronger numeric loop performance observed in earlier benchmarks.

The graphs show a notable step change in MT Wasm curves at approximately $n=2,000$ (less visible in Firefox due to the noise but its there), where rayon decides to start using threads for the workload, suddenly adding some overhead but giving a better scaling that will eventually defeat ST wasm at around 6-7k elements. Although having to sort such a large number of elements in the frontend is uncommon, suggesting MT wasm has none or even negative benefits for sorting workloads.

The sort benchmark additionally incurs a per-call `Vec<f64>` ownership copy (unlike reference-passing in other benchmarks), compounding the overhead. The Firefox graph (Figure 14) shows the most spiky curves that are likely caused by the coarser precision of performance.now in SpiderMonkey and the added jitter.

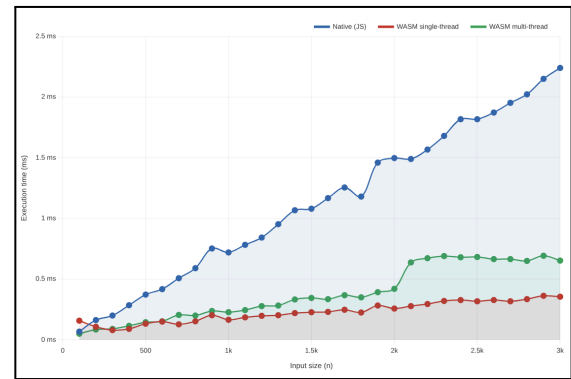


Fig. 13. Sort Chrome. Note the point at which rayon switches strategy.

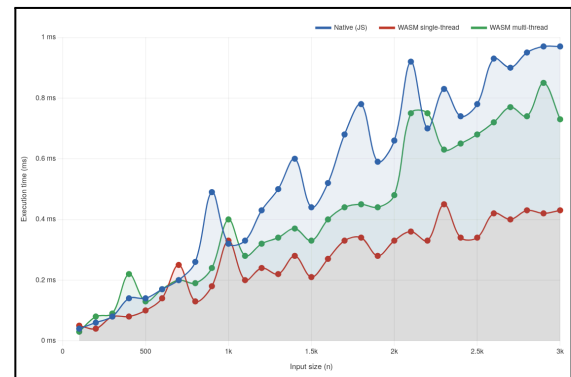


Fig. 14. Sort Firefox. Exactly $1.00\times$ speedup: multi-thread provides no benefit. Note the visible spikes caused by the limited precision and additional jitter introduced by SpiderMonkey in time measurements.

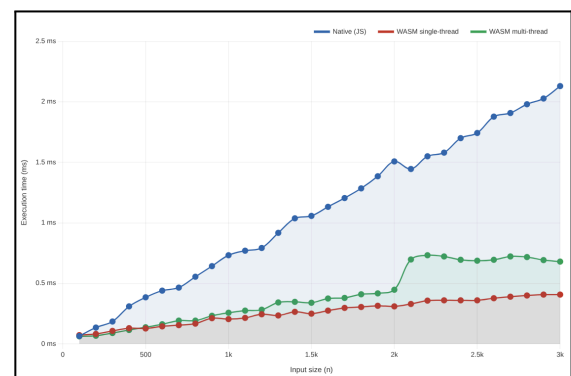


Fig. 15. Sort Microsoft Edge. Identical to Chrome

G. Correlation: Memory Bandwidth Saturation

Correlation exhibits similar multi-thread underperformance to sort, but for a different reason. Pearson correlation is memory-bandwidth-bound: it makes multiple sequential passes over two input arrays with low arithmetic intensity (few operations per byte loaded). Multiple threads accessing the same array simultaneously contend for the shared memory bus and may reduce effective bandwidth per thread. That said, the multi-thread curve shows better scaling slope than single-thread Wasm across all browsers, and the two lines are converging suggesting that at input sizes well beyond the tested range, MT would eventually overtake single-thread once the per-element work outweighs the coordination cost. Though once again, workload that large is uncommon in the frontend. Therefore MT provides a negative benefit for lightweight tasks.

All three implementations use functional combinators for the computation map, reduce, and equivalent operations but with a critical semantic difference. JavaScript's array methods are eager, each function call allocates a full copy of the array before the next step begins, producing several large heap allocations per benchmark call. Rust's iterator combinators are lazy and zero-allocation, composing into a single pass with no intermediate heap use. This difference shows up directly in the Firefox graph (Figure 18), where native JS shows several large isolated spikes absent from both Wasm curves. Because all three share the same performance.now clock and executed sequentially for each input, any external cause would produce coincident spikes across all three lines. The absence of spikes in wasm rules this out and identifies the cause as specific JavaScript GC pauses or JIT behaviour in SpiderMonkey.

Figures 16-18 reveal an interesting pattern: single-thread Wasm (red) significantly outperforms native JS (blue) in all browsers, with Chrome showing approximately a 12× advantage at n=29,000. This is the largest JS-to-single-thread-Wasm gap observed across all five benchmarks wider than even matrix multiplication and image blur, which are computationally heavier workloads. The reason is that those heavier benchmarks are bottlenecked by arithmetic or memory bandwidth, where V8's JIT can at least partially close the gap through loop optimisation and vectorisation. Correlation, by contrast, suffers the double penalty of GC pressure from eager intermediate allocations and the JIT's inability to elide them costs that simply do not exist in Rust's lazy zero-allocation iterator pipeline. LLVM can therefore emit a tight, fully unrolled, auto-vectorised single pass, while the JS engine is spending a significant

fraction of its time in GC rather than doing arithmetic. The result is that the performance gap here reflects not just code generation quality but a fundamental difference in evaluation strategy between the two languages.

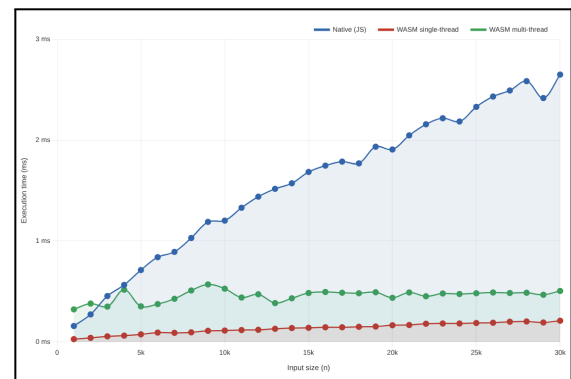


Fig. 16. Correlation Chrome. Single-thread Wasm (red) is faster than native JS (blue), demonstrating LLVM's superior loop optimization.

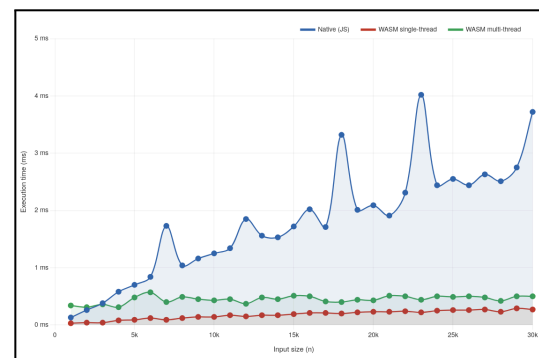


Fig. 17. Correlation Firefox. The native JS line (blue) shows characteristic variability spikes from JIT events and GC pauses at larger input sizes. The lines look closer because the largest spike touches 4ms, leading to the y axis going up to 5ms instead of 3ms.

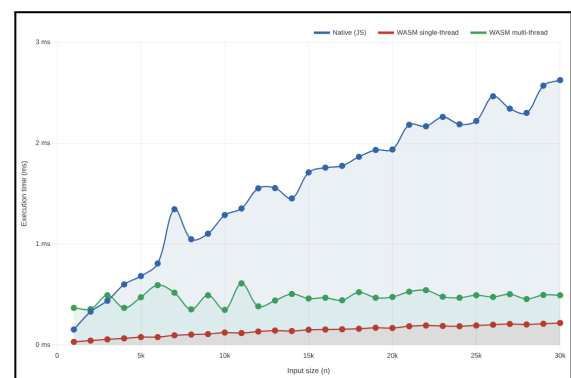


Fig. 18. Correlation Microsoft Edge. Pattern mirrors Chrome

V. DISCUSSION

A. Workload Selection

The visual evidence across Figures 1–18 and the data in Table I support a coherent framework for deciding when multithreaded Wasm is appropriate. The key variable is the ratio of arithmetic computation per element to inter-thread coordination cost per element. When this ratio is high as in matrix multiplication and image blur, threading approaches linear speedup. When it is low as in sort, grep, and correlation, threading provides zero or negative benefit.

This ratio is not strictly determined by big-O complexity class. Image blur is $O(n^2)$ and scales well; sort is $O(n \log n)$ and does not at tested input sizes. Embarrassingly parallel workloads with high arithmetic intensity are strong candidates; coordination-heavy or memory-bandwidth-limited workloads are not. Table II summarizes this framework.

TABLE II. WORKLOAD CHARACTERIZATION AND WASM THREADING RECOMMENDATIONS

Test	Complexity	MT Speedup	Workload type	Recommendation
Matrix Multiply	$O(n^3)$	~ 8	Compute Heavy	Strong MT candidate
Image Blur	$O(n^2)$	~ 3	Arithmetic	Good MT candidate
Grep	$O(n)$	< 1	String handling	Use native JS, avoid wasm
Sort	$O(n \log n)$	≤ 1	Coordination	Use ST wasm
Correlation	$O(n)$	< 1	Lightweight, iterator based	Use ST wasm

B. The JS $\leftrightarrow$ Wasm Boundary Cost

The grep results (Figs. 10–12) demonstrate the magnitude of the string boundary cost. UTF-16 $\rightarrow$ UTF-8 transcoding and memory copy per call is sufficient to make any Wasm variant slower than native JavaScript for string operations. Two mitigation strategies exist:

1. Preload string data into Wasm linear memory once and perform all processing within Wasm.
2. Use enums instead of strings wherever possible to avoid conversions entirely.

For numeric data, the situation is far more favorable: f64 arrays can be passed as zero-copy pointer+length references, as demonstrated by the correlation benchmark's strong single-thread Wasm advantage

C. Cross-Browser Consistency and Divergence

The qualitative pattern that matrix multiply and image blur benefit from threading; grep, sort, and correlation do not is consistent across all three browsers, confirming architectural rather than engine-specific causation.

The variability characteristics visible in the graphs are also engine-dependent. Firefox's native JS curves for sort (Fig. 14) and correlation (Fig. 17) show more frequent and larger spikes than Chrome/Edge, consistent with SpiderMonkey's garbage collector having different pausing behavior than V8's incremental GC. These spikes are precisely why median rather than mean is used as the primary metric. For most tests Firefox also showed more noise partly due to the poorer reliability of performance.now.

The two engines also diverge in JIT architecture in ways that explain several of the performance differences observed across benchmarks. V8 uses a two-tier JIT: Sparkplug (a fast non-optimising baseline compiler) and TurboFan (a highly optimising compiler with sea-of-nodes IR). TurboFan is heavily tuned for polymorphic and object-heavy workloads typical of general web applications. SpiderMonkey uses WarpMonkey as its optimising tier, built on a CacheIR-based frontend that specialises aggressively on observed types. For the dense numeric loops characteristic of matrix multiplication image blur and sort, WarpMonkey's type specialisation produces tighter arithmetic code than TurboFan, which is reflected in SpiderMonkey's consistently lower native JS times on those benchmarks. Conversely, both engines produce nearly identical Wasm performance across all benchmarks, since Wasm's static types remove the need for speculative optimisation and both engines' Wasm compilers target the same LLVM-compiled output at similar quality levels.

D. Implications for Application Architecture

These results have direct implications for web application architecture. For matrix operations (neural network layers, scientific visualization, WebGL preprocessing), multithreaded Wasm provides near-linear scaling and is a strong choice—an 8× speedup means a 24 ms operation completes in under 3 ms. For image processing pipelines operating on large images, multithreaded Wasm provides 3× speedup, meaningful for real-time video applications. For string processing (search, parsing, text analysis), native JavaScript is categorically preferable unless data can remain in Wasm memory. For sorting at typical web scales (tens to hundreds to a few thousands of elements), single-thread Wasm is the intuitive choice and MT does not provide as much benefit as it suffers from overhead. Rust’s zero cost and lazy iterator implementations are also available in wasm, making it ideal for iterator-like data pipelines.

E. Threats to Validity

Limitations include:

- Tests performed on a single machine with a single chosen thread count, results may differ on 4-core or 16-core systems.
- Browser and wasm version. Wasm is evolving rapidly and is expected to get better with time. Browsers may as well improve support for the SharedArrayBuffer leading to even better performance and lower overhead.
- Constrained input size ranges. Sort in particular might show different results at 100,000-element inputs although these sizes are rare in the frontend.
- Uncontrolled background processes contributing to the observed high CVs. These do not affect the qualitative conclusions, but caution should be applied when extrapolating specific speedup numbers.
- Instability in the wasm-bindgen ecosystem (at the time of writing this paper). With change in ownership, the strategies used by the crate to handle js-wasm interactions may change.
- Limited javascript engines tested, results may vary on other browsers such as safari (JavaScriptCore).

VI. FUTURE WORK

Several directions merit future investigation:

- A thread-count scalability analysis would establish speedup curves as thread count varies 1–16, identifying diminishing-returns thresholds for matrix multiply and image blur.
- SIMD evaluation would quantify the additional benefit of Wasm’s 128-bit SIMD instructions for arithmetic workloads.
- A broader benchmark suite including cryptography, audio processing (FFT), graph algorithms, object or polymorphic workload, and ML inference would provide wider coverage of the client-side compute landscape.
- Evaluation of multithreading libraries in native js. A few exist already, such as multithreading and atomics-sync on npm.js, but these are far from popular or mature, having less than 1500 downloads combined. Once a library with a library with enough maturity is available, it can be added to the comparison.

VII. CONCLUSION

This paper presented a systematic empirical evaluation of single-threaded and Rayon-based multithreaded WebAssembly compiled from Rust, compared against native JavaScript, across six benchmark tasks and three major browsers. Eighteen execution-time graphs across all browser/benchmark combinations provide a comprehensive visual characterization of runtime behavior. The central findings are:

1. AOT Compiler Advantage: LLVM’s constant folding reduced single-thread Wasm sum execution to zero—a capability structurally impossible for browser JIT compilers.
2. Strong Parallel Scaling for Compute-Intensive Workloads: Multithreaded Wasm achieves ~8x speedup for matrix multiplication and 3.1–3.4× for image blur on 8 threads, approaching the theoretical linear maximum.
3. Performance Inversion for String Workloads: The UTF-16→UTF-8 transcoding cost at the JS↔Wasm boundary makes native JavaScript faster than any Wasm variant for per-call string operations, as demonstrated across all three browsers.
4. No Benefit for Memory-Bound or Small-Input Sort: Multithreaded Wasm provides zero or negative returns for correlation (memory bandwidth saturation) and sort at tested input

sizes (below Rayon's ~100,000-element break-even threshold).

5. Cross-Browser Consistency: Qualitative patterns are consistent across Firefox, Chrome, and Edge, confirming architectural causation. Quantitative differences are meaningful and browser-specific.

In summary, multithreaded Wasm is not a universal solution, but a powerful, specialized tool. Its effective use depends on careful workload characterization, thoughtful system design, and continued advances in both browser engines and developer tooling.

REFERENCES

- [1] WebAssembly Community Group, "WebAssembly features status," 2025. [Online]. Available: <https://webassembly.org/>
- [2] "wasm-bindgen," Rust Package Registry (crates.io). [Online]. Available: <https://crates.io/crates/wasm-bindgen>
- [3] "Rayon," Rust Package Registry (crates.io). [Online]. Available: <https://crates.io/crates/rayon>
- [4] "wasm-bindgen-rayon," Rust Package Registry (crates.io). [Online]. Available: <https://crates.io/crates/wasm-bindgen-rayon>
- [5] "asm.js: Typed subset of JavaScript." [Online]. Available: <http://asmjs.org/>
- [6] "Emscripten: C/C++ to WebAssembly Compiler Documentation." [Online]. Available: <https://emscripten.org/>
- [7] WebAssembly, "Feature Status." [Online]. Available: <https://webassembly.org/features/>
- [8] "Wasmtime: WebAssembly Runtime." [Online]. Available: <https://wasmtime.dev/>
- [9] "Wasmer: Universal WebAssembly Runtime." [Online]. Available: <https://wasmer.io/>
- [10] B. Spies and M. Mock, "An Evaluation of WebAssembly in Non-Web Environments," in Proc. XLVII Latin American Computing Conference (CLEI), Cartago, Costa Rica, 2021, pp. 1–10. doi: 10.1109/CLEI53233.2021.9640153
- [11] G. Perrone and S. P. Romano, "WebAssembly and security: A review," Computer Science Review, vol. 56, 2025, Art. no. 100728. doi: 10.1016/j.cosrev.2025.100728
- [12] MDN Web Docs, "Using Web Workers API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers
- [13] MDN Web Docs, "SharedArrayBuffer." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer
- [14] "Spectre Attack: Exploiting Speculative Execution," ACM Digital Library. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3399742>